Stellar / Exoplanet Prediction Model Submission

Project Title: Stellar Analytics
Team Name: EV+

Team Members:

- Sneha Agrawal
- Suraj Kumar

Date: 4 March 2026

Contact Information:

- sneha15sep04@gmail.com
- surajkumargupta0901@gmail.com

---

1. Problem Statement

The Challenge of False Signals

The discovery of exoplanets has transformed our understanding of the universe. However, not every detected signal corresponds to a real planet. Astronomical observations frequently contain noise and misleading signals caused by binary star systems, stellar variability, instrumental noise, and background eclipsing systems. These phenomena often mimic the transit signals produced by planets.

As a result, astronomers face a fundamental challenge:

Which detected signals represent real exoplanets, and which are false positives?

Even after identifying a potential planet, several planetary characteristics—such as planetary radius—must still be estimated from incomplete and noisy observational data. These estimates rely heavily on stellar parameters and transit signal measurements.

Mission Brief: The Stellar Verification Program

This project contributes to the **Stellar Verification Program**, an initiative aimed at improving the reliability of exoplanet detection through data-driven approaches.

Using a merged dataset containing **Kepler Objects of Interest (KOIs)** along with host star parameters, the objective of this project is twofold:

Classification Task

Distinguish between **confirmed exoplanets** and **false positives** using machine learning models trained on stellar and transit features.

Regression Task

Accurately **predict the planetary radius (`koi_prad`)** of confirmed exoplanets using regression models trained on astronomical observation data.

The final system provides:

- Planet classification (**Confirmed / False Positive**)
- Prediction **confidence score**

- Estimated **planetary radius**

This approach contributes to improving automated exoplanet detection pipelines and enhances our understanding of planetary systems beyond our solar system.

---

2. Assumptions

- The dataset (`supernova_dataset.csv`) contains reliable observational data from the Kepler mission.
- Non-astronomical identifiers such as `kepid` do not contribute predictive value.
- Machine learning models can learn relationships between stellar parameters, transit signals, and planetary properties.
- The system operates in a modular architecture consisting of frontend, backend API, and ML models.

---

3. ML Tasks Description

Classification Task

A **binary classification problem** where the model predicts whether a signal corresponds to:

- **CONFIRMED Exoplanet**
- **FALSE POSITIVE**

Key features include:

- Transit depth
- Signal-to-noise ratio
- Transit duration
- Stellar temperature
- Orbital parameters

Regression Task

A regression model predicts **planetary radius (`koi_prad`)** for confirmed exoplanets.

To handle skewness in the radius distribution, the target variable is transformed using: `log1p(koi_prad)`

---

4. Approach

Data Preprocessing and Feature Engineering

- Loaded dataset using **Pandas**
- Performed **Exploratory Data Analysis (EDA)**
- Handled missing values
- Removed identifiers such as `kepid`
- Created separate datasets for classification and regression

Data Pipelines

Preprocessing pipelines were built using **Scikit-learn**:

- Missing values handled using `SimpleImputer(strategy="median")`
- Feature scaling using `StandardScaler()`

Train-Test Split

Dataset split:

- **80% Training**
- **20% Testing**

Stratified sampling maintained class balance.

Handling Missing or Unavailable Features

In real observational scenarios, some features may not always be available at prediction time. To ensure the model remains robust under such conditions, we evaluated the regression model by removing one important feature.

Removed Feature: `koi_ror`

This feature represents the ratio of planetary radius to stellar radius and may not always be available during early observation stages. The regression model was retrained without this feature to test the model's dependency on it.

Regression Performance Without `koi_ror`

Metric	Value
RMSE	1.66
MAE	0.314

Comparison With Original Model

Model	RMSE	MAE
With `koi_ror`	1.09	0.195
Without `koi_ror`	1.66	0.314

Interpretation

The results indicate that `koi_ror` is a highly informative feature for predicting planetary radius. However, even without this feature, the model maintains reasonable predictive capability.

Additionally, the preprocessing pipeline uses **SimpleImputer**, allowing the system to handle missing values gracefully during real-time inference.

This ensures the system remains **robust and deployable in real-world astronomical scenarios where incomplete data is common**.

---

5. Model Selection, Training, and Evaluation

Classification Models

Logistic Regression (Baseline)

Configuration:

- `class_weight = balanced`
- `max_iter = 1000`

Evaluation metrics: Precision, Recall, F1-score, ROC-AUC

Random Forest Classifier (Final Model)

Configuration:

- `n_estimators = 300`
- `max_depth = None`
- `class_weight = balanced`

Classification Model Performance

Class	Precision	Recall	F1-score	Support
False Positive (0)	0.93	0.96	0.95	968
Confirmed Planet (1)	0.93	0.87	0.90	549

Accuracy: 0.93
ROC-AUC: 0.981
This indicates excellent separation between confirmed planets and false positives.

Regression Model

XGBoost Regressor

Configuration:

- `n_estimators = 500`
- `learning_rate = 0.05`
- `max_depth = 6`
- `subsample = 0.8`
- `colsample_bytree = 0.8`

Regression Model Performance

Metric	Value
RMSE	1.09
MAE	0.195

Interpretation:

- Average prediction error ≈ **1 Earth radius**
- Predictions remain close to true radius values

System Performance

The system was also evaluated for engineering performance:

- Reliable **input validation**
- Fast **real-time inference**
- Robust **API communication**
- Clear **prediction outputs**
- Interpretable results via **feature importance**

---

6. System Architecture

Frontend

- Built with **React + Vite**
- Accepts user input
- Displays predictions and history

Backend

- Built using **FastAPI**
- Performs preprocessing and inference
- Provides REST API

Database

Supabase (PostgreSQL) stores:

- Input parameters
- Prediction results
- Timestamp logs

Prediction Workflow

1. User enters input parameters
2. Data sent to `/predict` API
3. Backend preprocesses data
4. ML models generate predictions
5. Results stored in database
6. Predictions returned to frontend

---

7. Results & Insights

The system generates three outputs:

1. **Planet Classification:** Confirmed Planet or False Positive
2. **Confidence Score:** Probability of prediction
3. **Planet Radius Prediction:** Estimated planetary radius in Earth radii

Feature Importance

Random Forest feature importance highlights the most influential stellar and transit features affecting exoplanet detection. These insights align with known astrophysical relationships.

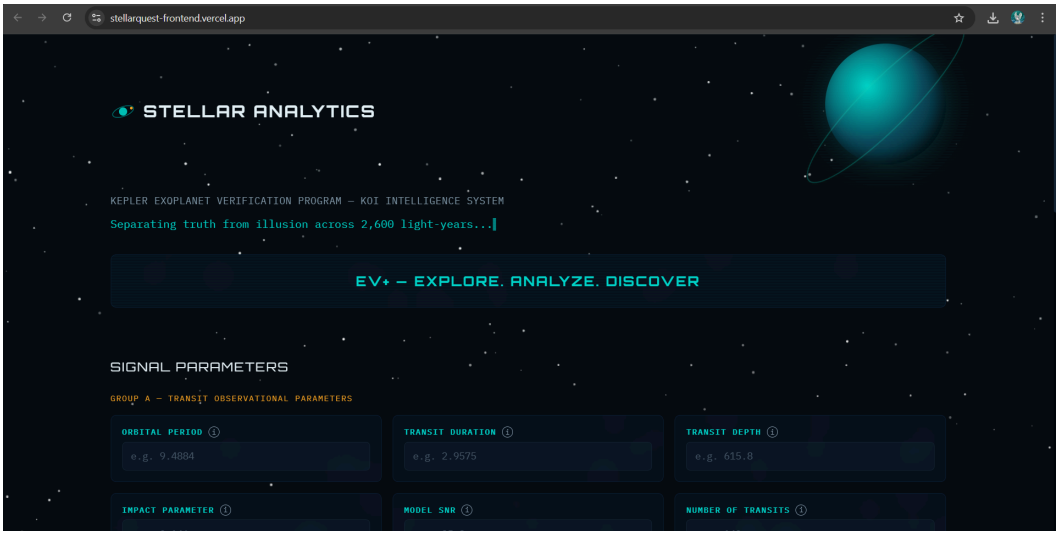
Appendix

Live Deployment

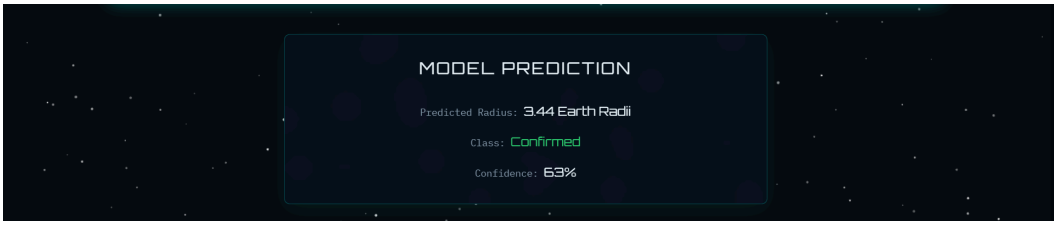
You can access the live version of our application here: [Stellarquest Analytics Platform]

System Interfaces & Visualizations

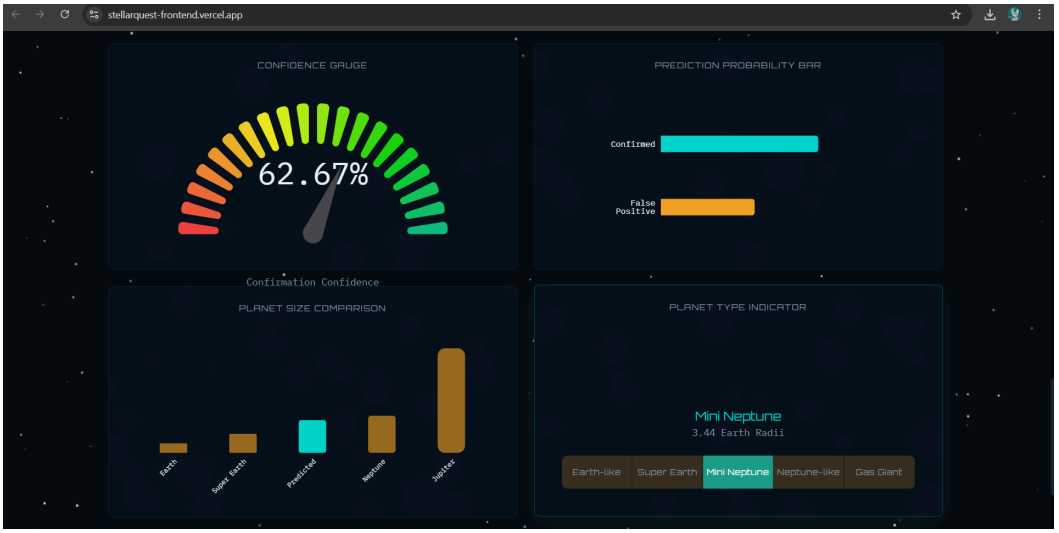
1. **Stellar Analytics Dashboard Interface**



2. **Model Prediction Result**



3. **Analytics Graphs & Confidence Visualization**



4. **Prediction History Logs**

MISSION LOG — PREDICTION HISTORY

TIMESTAMP	ORBITAL PERIOD	STELLAR TEMP	DISPOSITION	PRED. RADIUS	CONFIDENCE
3/5/2026, 10:44:15 AM	9.844	-	CONFIRMED	3.44 R⊕	62.7%
3/5/2026, 10:36:45 AM	904884	-	FALSE POSITIVE	3.03 R⊕	31.7%
3/5/2026, 9:23:56 AM	9.4884	-	FALSE POSITIVE	13.36 R⊕	17.7%
3/5/2026, 9:10:32 AM	1.025	-	FALSE POSITIVE	2.57 R⊕	24.0%
3/5/2026, 9:10:05 AM	9.484	-	CONFIRMED	2.54 R⊕	58.7%

< Previous 1 2 3 4 Next >

[STELLER ANALYTICS -IIT BHU TECHNEX 2026]